

TripVault

Travel Memory & Exploration Tracker

Week 3 – Trip Photo Uploads (Cloudinary) & Public User Profiles

A Project Report

NAME : KARTHIK R

Submitted as part of
TripVault Virtual Internship Program
Powered by CodGen

Technology Stack: React.js (Vite) • Node.js • Express.js • MongoDB Atlas • Mongoose • Cloudinary • Multer • JWT •
bcrypt.js

1. Abstract

This report documents the third milestone of the TripVault Virtual Internship Program, powered by CodGen. Weeks 1 and 2 delivered a secure authentication system and a complete Trip Management CRUD workflow. Week 3 extends both of those foundations in two directions: cloud-based photo storage for trips, and a public-facing user profile that lets a traveller's journeys be shared beyond their own logged-in session.

On the storage side, trip photo uploads move from local disk storage to Cloudinary, a managed media hosting service. A new upload configuration combines Multer with `multer-storage-cloudinary` so that images uploaded through the API are streamed directly to Cloudinary and referenced by URL, rather than being written to the server's own filesystem. The Trip model gains a `coverImage` field for a trip's primary photo and a `photos` array to support multiple images per trip in future, and a new `POST /api/trips/:id/upload` route lets a photo be attached to an existing trip after the fact.

On the profile side, the User model gains `username` and `bio` fields. A new `/api/users` route group exposes a public `GET /api/users/:username/profile` endpoint — accessible without authentication — that returns a user's name, bio, and trip list while deliberately excluding sensitive fields such as email and password, alongside a protected `PUT /api/users/profile` endpoint that lets the logged-in user update their own username or bio. Because usernames did not exist before this milestone, existing users are given a default username derived from their email prefix the next time they register or log in, keeping the new unique/required constraint on the User schema compatible with accounts created in Weeks 1 and 2.

On the frontend, the registration form gains a username field, the Dashboard gains “My Profile” and “Edit Profile” actions, and a new public PublicProfile page — reachable at `/profile/:username` without needing to be signed in — renders a user's name, bio, and a grid of their trip cards. This report walks through the objectives and scope of Week 3, the updated requirements, architecture, and database design, the new API surface, the folder structure, and the implementation details for both backend and frontend. It documents the algorithms behind username auto-generation, public profile retrieval, and photo upload, presents flowcharts for the profile and upload flows, and includes screenshots of the working application, the Swagger API documentation, and the underlying MongoDB collections as evidence of a functioning implementation.

2. Table of Contents

1. Abstract
2. Table of Contents
3. List of Figures
4. List of Tables
5. Recap of Weeks 1 & 2
6. Week 3 Objectives
7. Scope of Week 3
8. Functional Requirements
9. Non-Functional Requirements
10. Software Requirements
11. System Architecture
 - o Layer-by-Layer Explanation
 - o Public Profile & Photo Upload Workflow
12. Database Design
13. API Endpoints
14. Authentication & Authorization Recap
15. Folder Structure
16. Implementation
 - o Cloudinary Upload Configuration
 - o User Model Changes
 - o Username Auto-Generation
 - o Users Controller & Routes
 - o Trip Model Changes & Photo Upload Route
 - o Frontend — Registration, Dashboard, Public Profile
 - o Testing APIs
17. Component Screenshots
 - o Frontend
 - o API Documentation (Swagger)
 - o Database
18. Algorithms
19. Flowcharts
20. Results
21. Advantages
22. Future Enhancements
23. Conclusion
24. References
25. Appendix

3. List of Figures

Figure 1: System Architecture Diagram (Week 3).....	
Figure 2: Public Profile & Photo Upload Workflow Diagram	
Figure 3: Username Auto-Generation Flowchart.....	
Figure 4: Public Profile Retrieval Flowchart.....	
Figure 5: Trip Photo Upload Flowchart.....	
Figure 6: TripVault Frontend — Login Page	
Figure 7: TripVault Frontend — Dashboard (My Profile / Edit Profile actions).....	
Figure 8: TripVault Frontend — Dashboard Recent Highlights.....	
Figure 9: TripVault Frontend — World Exploration & Travel Statistics	
Figure 10: TripVault Frontend — Travel Journal (Memories).....	
Figure 11: TripVault Frontend — New Memory Form.....	
Figure 12: TripVault Frontend — Trip Planner (Trip List).....	
Figure 13: TripVault Frontend — Plan New Trip Form	
Figure 14: TripVault Frontend — Profile Page (bio, avatar upload, stats)	
Figure 15: Swagger — API Endpoint Overview (Users, Trips, Memories, Auth)	
Figure 16: Swagger — PUT /api/users/profile (Update Username / Bio).....	
Figure 17: Swagger — GET /api/users/{username}/profile (Public Profile)	
Figure 18: Swagger — GET /api/trips Response	
Figure 19: Swagger — GET /api/trips/{id} Response.....	
Figure 20: Swagger — POST /api/trips (Create Trip)	
Figure 21: Swagger — DELETE /api/trips/{id} Response	
Figure 22: Swagger — GET /api/memory Response	
Figure 23: Swagger — POST /api/memory (Create Memory).....	
Figure 24: Swagger — DELETE /api/memory/{id} Response	
Figure 25: Swagger — POST /api/auth/register	
Figure 26: Swagger — POST /api/auth/login (credentials redacted)	
Figure 27: Swagger — GET /api/auth/me Response.....	
Figure 28: MongoDB — trips Collection.....	
Figure 29: MongoDB — memories Collection.....	

4. List of Tables

Table 1: User Collection Schema (Updated).....	
Table 2: Trip Collection Schema (Updated).....	
Table 3: Users & Trips API Endpoints (Week 3 additions).....	
Table 4: Functional Requirements Summary (Week 3).....	
Table 5: Software Additions for Week 3.....	

5. Recap of Weeks 1 & 2

Week 1 established TripVault's technical foundation: a React.js (Vite) frontend, a Node.js/Express.js backend, MongoDB Atlas, and JWT-based authentication with bcrypt-hashed passwords. Week 2 built on that foundation with a complete Trip Management CRUD feature — a Trip model, five protected `/api/trips` endpoints, local Multer-based photo storage, and a React Trip Planner UI, alongside a parallel Memories module.

Week 3 does not change the authentication system itself; every new route introduced here either sits behind the same JWT middleware, or — in the case of the public profile — is deliberately left open, with sensitive fields filtered out at the controller level instead of behind a login wall.

6. Week 3 Objectives

- To move trip photo storage from the local filesystem to Cloudinary, a managed cloud media host.
- To configure Multer with `multer-storage-cloudinary` so uploads stream directly to Cloudinary.
- To add `coverImage` and `photos` fields to the Trip model in preparation for richer trip galleries.
- To add a POST `/api/trips/:id/upload` route that attaches a photo to an already-created trip.
- To add `username` and `bio` fields to the User model, with `username` unique and required.
- To auto-generate a default username (from the email prefix) for users who registered before this milestone.
- To build a public GET `/api/users/:username/profile` endpoint that never exposes email or password.
- To build a protected PUT `/api/users/profile` endpoint so a user can edit their own username and bio.
- To add a `username` field to the registration form and “My Profile” / “Edit Profile” actions to the Dashboard.
- To build a public `PublicProfile` page, reachable without logging in, showing a user's bio and trip grid.

7. Scope of Week 3

The scope of Week 3 covers two related features: cloud-hosted trip photos and public user profiles. On the backend this means a new Cloudinary-backed upload configuration, schema changes to both the User and Trip models, a new users route group, and a new trip photo upload route. On the frontend it means an updated registration form, an updated Dashboard, and a new, unauthenticated `PublicProfile` page.

Deeper photo-gallery features — multiple photos per trip surfaced in the UI, image cropping, or per-photo captions — are intentionally out of scope for this milestone; the `photos` array on the Trip model lays the groundwork for them but only a single cover image is wired up end-to-end this week. As before, richer analytics, maps, and AI recommendations remain future work.

8. Functional Requirements

The functional requirements below describe the behaviours the Week 3 features must support.

Requirement	Description
Cloud Photo Storage	Trip photos uploaded through the API are stored on Cloudinary and referenced by URL, not on local disk.
Attach Photo to Trip	Allow a logged-in user to upload a photo to one of their existing trips via POST /api/trips/:id/upload.
Username on Registration	Allow a new user to choose a username at registration, enforced unique.
Username Backfill	Automatically generate a default username (from the email prefix) for a pre-existing user who logs in without one.
Public Profile Read	Allow anyone, signed in or not, to view a user's name, bio, and trip list by username.
Sensitive Field Exclusion	Never return a user's password or email from the public profile endpoint.
Profile Self-Edit	Allow the logged-in user to update their own username or bio via a protected endpoint.
Public Profile Page	Render a public, no-login-required profile page in the React app at /profile/:username.

Table 4: Functional Requirements Summary (Week 3)

9. Non-Functional Requirements

Security

Cloudinary credentials (cloud name, API key, API secret) are read from environment variables and never committed to source control; the public profile endpoint is carefully scoped to exclude password and email at the query level, not just at the response-serialisation level.

Privacy

A user's public profile only ever exposes what they chose to make public — name, bio, and trip summaries — never account credentials or contact details.

Reliability

Photo uploads should fail gracefully: if Cloudinary is unreachable or credentials are misconfigured, trip creation or update should still succeed for the text fields, with a clear error surfaced for the image portion.

Backward Compatibility

Existing users created in Weeks 1–2, before the username field existed, must be able to log in without being blocked by the new unique/required constraint — handled by the auto-generation fallback described in the objectives above.

Performance

Serving trip images from Cloudinary's CDN, rather than from the Node.js server's own disk, keeps image delivery fast and off the application server's request path.

Maintainability

The new upload.js, users.js, and their associated model changes follow the same modular MVC-inspired structure established in Weeks 1 and 2.

10. Software Requirements

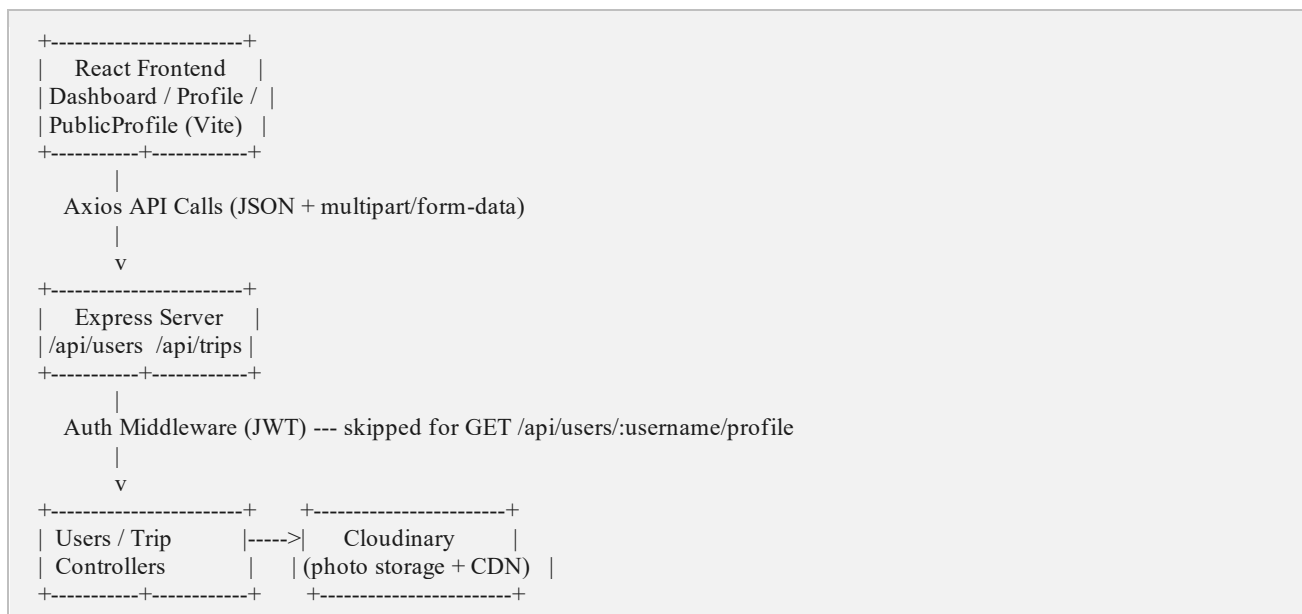
Week 3 reuses the full stack from Weeks 1–2 (React.js/Vite, Node.js, Express.js, MongoDB Atlas, Mongoose, JWT, bcrypt.js, Multer) and adds the following:

Software / Library	Purpose
Cloudinary	Cloud-based image and video hosting used to store trip photos and serve them via CDN URL
cloudinary (npm)	Official Node.js SDK used to configure the Cloudinary connection from environment variables
multer-storage-cloudinary	Multer storage engine that streams uploaded files directly to Cloudinary instead of local disk
.env additions	CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, and CLOUDINARY_API_SECRET added to the server environment

Table 5: Software Additions for Week 3

11. System Architecture

Week 3 introduces a new external dependency — Cloudinary — sitting alongside MongoDB Atlas in the backend's architecture. Photo uploads for trips no longer stop at the Express server's own disk; Multer, configured with the multer-storage-cloudinary storage engine, streams the file straight through to Cloudinary, and only the returned URL is persisted on the Trip document. The public profile route is the first endpoint in the project that deliberately sits outside the JWT authentication middleware, while still being served by the same Express application and controllers.



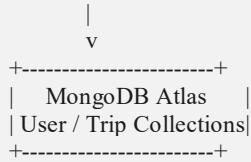


Figure 1: System Architecture Diagram (Week 3)

Layer-by-Layer Explanation

React Frontend

The Dashboard now surfaces “My Profile” and “Edit Profile” actions, and a new PublicProfile page renders a shareable, read-only view of a user's trips at `/profile/:username`.

Axios API Calls

Profile edits are sent as JSON to `PUT /api/users/profile` with the JWT attached; public profile reads are plain unauthenticated GET requests; trip photo uploads use `multipart/form-data` as they did in Week 2.

Authentication Middleware

Every route continues to use the Week 1 JWT middleware except `GET /api/users/:username/profile`, which is intentionally public so a profile can be viewed or shared without an account.

Users / Trip Controllers

The users controller assembles the public profile response by explicitly selecting only name, bio, username, and trip summary fields — never querying or returning the password or email fields. The trip controller's new upload handler attaches the Cloudinary URL returned by Multer to the trip's `coverImage` field.

Cloudinary

Cloudinary receives the uploaded file directly from Multer's storage engine, stores it, and returns a hosted URL and metadata that the controller saves on the relevant document.

MongoDB Atlas

The User and Trip collections gain new fields (username, bio, coverImage, photos) but keep the same overall document shape and ownership model established in Weeks 1–2.

Public Profile & Photo Upload Workflow

```

PHOTO UPLOAD
[User] --(selects photo)--> [React: Trip Form / Upload]
|
v
[Axios: multipart/form-data + JWT] --> [POST /api/trips/:id/upload]
|
[Auth Middleware: verify JWT] --> [Multer + Cloudinary storage]
|
[Trip Controller: save Cloudinary URL to coverImage]
|
[MongoDB Atlas: update Trip document]

PUBLIC PROFILE
[Visitor] --(opens /profile/:username)--> [React: PublicProfile]
  
```

```

|
v
[Axios: GET, no auth header] --> [GET /api/users/:username/profile]
|
[Users Controller: find user by username]
|
[Select name, bio, trips only -- exclude email & password]
|
[Response: public profile JSON] --> [React renders bio + trip grid]

```

Figure 2: Public Profile & Photo Upload Workflow Diagram

12. Database Design

Week 3 extends both core collections introduced in Weeks 1–2 rather than adding a new one.

Field	Type	Description
_id	ObjectId	MongoDB-generated unique identifier for each user document.
name	String	Full name of the user, provided at registration. (Week 1)
email	String	Unique login identifier. (Week 1)
password	String	bcrypt-hashed password. (Week 1)
username	String (unique, required)	New in Week 3. Public handle used in profile URLs; auto-generated from the email prefix for pre-existing accounts.
bio	String (optional)	New in Week 3. Free-text line shown on the public profile, e.g. “Travel enthusiast and writer.”
createdAt	Date	Timestamp automatically recorded when the account is created. (Week 1)

Table 1: User Collection Schema (Updated)

Field	Type	Description
_id	ObjectId	MongoDB-generated unique identifier for each trip document.
title / destination / startDate / endDate / description / rating	—	Unchanged from Week 2.
image	String	Legacy single-photo path from Week 2 (local/uploads path for trips created before Week 3).
coverImage	String	New in Week 3. Cloudinary URL of the trip's primary photo, set by POST /api/trips/:id/upload or by create/update with a file attached.
photos	[String]	New in Week 3. Array of Cloudinary URLs, reserved for future multi-photo gallery support.
user	ObjectId (ref: 'User')	Links the trip to its owner; used to scope every query. (Week 2)

Table 2: Trip Collection Schema (Updated)

Field Explanation

- **username:** enforced unique and required at the schema level, mirroring how email uniqueness was enforced in Week 1 — with a one-time backfill for accounts that predate this field.
- **bio:** intentionally optional and unstructured, giving users a short free-text line for their public profile.
- **coverImage** vs. **image:** **coverImage** is the Cloudinary-hosted successor to Week 2's local image field; both are kept so trips created before this milestone continue to render correctly.
- **photos:** left empty for now — the schema supports it, but only the frontend flows for a single cover image are wired up this week.

13. API Endpoints

The table below lists the endpoints added in Week 3. All Week 1 and Week 2 endpoints (`/api/auth/*`, `/api/trips`, `/api/memory`) remain unchanged and continue to work exactly as documented in the previous reports.

Method	Endpoint	Description
GET	<code>/api/users/:username/profile</code>	Public — get a user's name, bio, and trip list by username. No JWT required.
PUT	<code>/api/users/profile</code>	Protected — update the logged-in user's own username or bio.
POST	<code>/api/trips/:id/upload</code>	Protected — upload a photo for an existing trip; stores it on Cloudinary and saves the URL to <code>coverImage</code> .

Table 3: Users & Trips API Endpoints (Week 3 additions)

Note that GET `/api/users/:username/profile` is the project's first deliberately public route — every other endpoint, including the pre-existing `/api/trips` and `/api/memory` routes, still requires a valid JWT.

14. Authentication & Authorization Recap

Weeks 1 and 2 established a simple rule: every protected route requires a valid JWT, checked by the `protect` middleware, and `trip/memory` routes additionally check resource ownership. Week 3 adds a deliberate exception to that rule — GET `/api/users/:username/profile` is designed to be public, because a profile is only useful if it can be shared with people who do not have a TripVault account. Rather than weakening authentication, this endpoint keeps the security boundary in the right place: it is a read-only query that explicitly selects a small, safe subset of fields (name, bio, username, and trip summaries), so there is no path through it to a user's email, password hash, or any other user's private data. PUT `/api/users/profile`, by contrast, remains fully protected — a user can only ever edit their own username and bio, verified from the JWT's user ID, never from a value supplied in the request body.

15. Folder Structure

```
Tripvault/
|-- server/
```

```

|-- models/
| |-- User.js      (updated: + username, bio)
| |-- Trip.js      (updated: + coverImage, photos)
| |-- Memory.js
|-- routes/
| |-- authRoutes.js
| |-- tripRoutes.js (updated: + /:id/upload)
| |-- memoryRoutes.js
| |-- userRoutes.js (new)
|-- middleware/
| |-- authMiddleware.js
| |-- upload.js     (new: Cloudinary + Multer storage)
|-- controllers/
| |-- authController.js (updated: username handling)
| |-- tripController.js (updated: upload handler)
| |-- memoryController.js
| |-- userController.js (new)
|-- config/
| |-- db.js
| |-- cloudinary.js (new)
|-- .env             (+ CLOUDINARY * keys)
|-- index.js         (registers /api/users)
|-- client/
| |-- src/
| | |-- components/
| | |-- pages/
| | | |-- Register.jsx (updated: + username field)
| | | |-- Dashboard.jsx (updated: + My Profile / Edit Profile)
| | | |-- Trips.jsx
| | | |-- Memories.jsx
| | | |-- Profile.jsx
| | | |-- PublicProfile.jsx (new)
| | |-- App.jsx        (updated: + /profile/:username route)

```

Folder Explanation

- server/middleware/upload.js — new Cloudinary + multer-storage-cloudinary configuration, replacing the local-disk Multer setup used for trip photos in Week 2.
- server/config/cloudinary.js — initialises the Cloudinary SDK from the CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, and CLOUDINARY_API_SECRET environment variables.
- server/routes/userRoutes.js — maps the two new /api/users endpoints; only the PUT route is wrapped in the auth middleware.
- server/controllers/userController.js — implements getPublicProfile (field-limited, unauthenticated) and updateProfile (protected, self-only).
- client/src/pages/PublicProfile.jsx — the new, unauthenticated page rendered at /profile/:username.

16. Implementation

Cloudinary Upload Configuration

The Cloudinary SDK is initialised once from environment variables, then wrapped in a CloudinaryStorage engine that Multer uses in place of Week 2's local disk storage:

```
// config/cloudinary.js
const cloudinary = require('cloudinary').v2;

cloudinary.config({
  cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
  api_key: process.env.CLOUDINARY_API_KEY,
  api_secret: process.env.CLOUDINARY_API_SECRET,
});

module.exports = cloudinary;
```

```
// middleware/upload.js
const multer = require('multer');
const { CloudinaryStorage } = require('multer-storage-cloudinary');
const cloudinary = require('../config/cloudinary');

const storage = new CloudinaryStorage({
  cloudinary,
  params: {
    folder: 'tripvault/trips',
    allowed_formats: ['jpg', 'jpeg', 'png', 'webp'],
  },
});

module.exports = multer({ storage });
```

User Model Changes

username and bio are added to the existing User schema, with username marked unique and required:

```
// models/User.js (Week 3 additions)
username: {
  type: String,
  required: true,
  unique: true,
  trim: true,
},
bio: {
  type: String,
  default: "",
},
```

Username Auto-Generation

To keep the new required, unique constraint compatible with accounts created in Weeks 1–2, a helper derives a candidate username from the email prefix and appends a numeric suffix if that handle is already taken:

```
// controllers/authController.js (excerpt)
const generateUsernameFromEmail = async (email) => {
  const base = email.split('@')[0].toLowerCase().replace(/^[a-z0-9]/g, "");
  let candidate = base;
  let suffix = 0;
  while (await User.findOne({ username: candidate })) {
    suffix += 1;
  }
```

```

    candidate = `${base}${suffix}`;
  }
  return candidate;
};

// Called during register (if no username supplied) and during login
// for accounts created before this field existed.
exports.ensureUsername = async (user) => {
  if (user.username) return user;
  user.username = await generateUsernameFromEmail(user.email);
  await user.save();
  return user;
};

```

Users Controller & Routes

`getPublicProfile` explicitly selects only name, bio, and username from the User document — email and password are never even fetched from the database, let alone returned. `updateProfile` only ever edits the document belonging to `req.userId`, taken from the verified JWT:

```

// controllers/userController.js
const User = require('../models/User');
const Trip = require('../models/Trip');

exports.getPublicProfile = async (req, res) => {
  const user = await User.findOne({ username: req.params.username }).select('name bio username');
  if (!user) return res.status(404).json({ message: 'User not found' });

  const trips = await Trip.find({ user: user._id })
    .select('title destination startDate endDate rating coverImage')
    .sort({ startDate: -1 });

  res.status(200).json({ name: user.name, bio: user.bio, username: user.username, trips });
};

exports.updateProfile = async (req, res) => {
  const { username, bio } = req.body;
  const user = await User.findById(req.userId);

  if (username && username !== user.username) {
    const taken = await User.findOne({ username });
    if (taken) return res.status(400).json({ message: 'Username already taken' });
    user.username = username;
  }
  if (bio !== undefined) user.bio = bio;

  await user.save();
  res.status(200).json(user);
};

```

```

// routes/userRoutes.js
const router = require('express').Router();
const protect = require('../middleware/authMiddleware');
const { getPublicProfile, updateProfile } = require('../controllers/userController');

// Public — no auth middleware

```

```
router.get('/:username/profile', getPublicProfile);

// Protected — self only
router.put('/profile', protect, updateProfile);

module.exports = router;
```

Trip Model Changes & Photo Upload Route

coverImage and photos are added to the Trip schema, and a new route lets a photo be attached to a trip after creation, storing the Cloudinary-hosted URL that Multer attaches to req.file.path:

```
// controllers/tripController.js (new handler)
exports.uploadTripPhoto = async (req, res) => {
  const trip = await Trip.findOne({ id: req.params.id, user: req.userId });
  if (!trip) return res.status(404).json({ message: 'Trip not found' });
  if (!req.file) return res.status(400).json({ message: 'No image uploaded' });

  trip.coverImage = req.file.path; // Cloudinary URL supplied by multer-storage-cloudinary
  await trip.save();
  res.status(200).json(trip);
};

// routes/tripRoutes.js (addition)
// const upload = require('../middleware/upload'); // Cloudinary-backed
router.post('/:id/upload', protect, upload.single('image'), uploadTripPhoto);
```

Frontend — Registration, Dashboard, Public Profile

The registration form now collects a username alongside name, email, and password:

```
// pages/Register.jsx (excerpt)
<input
  name="username"
  placeholder="Choose a username"
  value={form.username}
  onChange={handleChange}
  required
/>
```

The new PublicProfile page fetches its data with a plain, unauthenticated GET request — it works identically whether or not the visitor is logged in:

```
// pages/PublicProfile.jsx (excerpt)
const { username } = useParams();
const [profile, setProfile] = useState(null);

useEffect(() => {
  // No Authorization header -- this call works whether or not the visitor is signed in.
  api.get(`/api/users/${username}/profile`).then(({ data }) => setProfile(data));
}, [username]);

return profile && (
```

```

    <h1>{profile.name}</h1>
    <p>{profile.bio}</p>
    <div className="trip-grid">
      {profile.trips.map((t) => <TripCard key={t. id} trip={t} readOnly />)}
    </div>
  </>
);

```

Testing APIs

The new `/api/users` endpoints and the trip photo upload route were tested through the project's Swagger UI alongside the full set of Week 1 and Week 2 endpoints. Testing confirmed that the public profile route returns a 200 with only the expected fields for a valid username, a 404 for an unknown one, and that the profile-update route rejects an attempt to claim a username that is already taken.

17. Component Screenshots

The following screenshots, captured from the working TripVault implementation, illustrate the Week 3 features — Cloudinary-backed trip photos and public user profiles — across the frontend, the API documentation, and the database.

Frontend

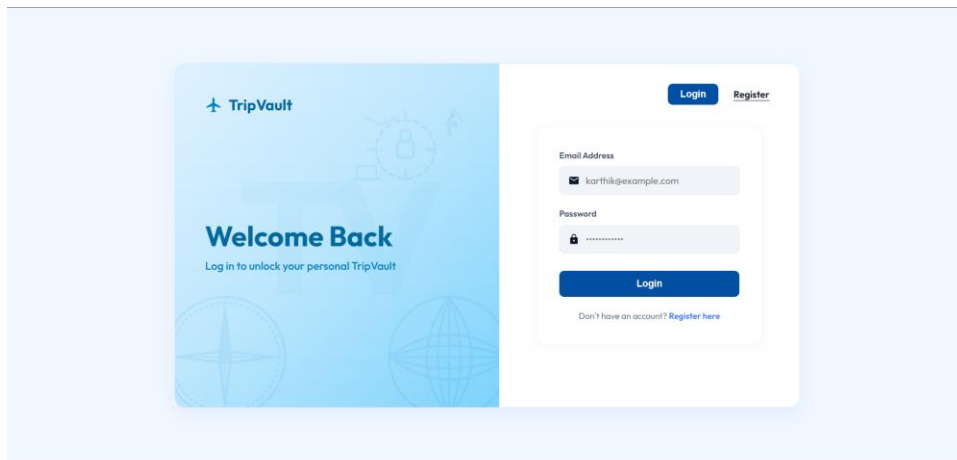


Figure 6: TripVault Frontend — Login Page

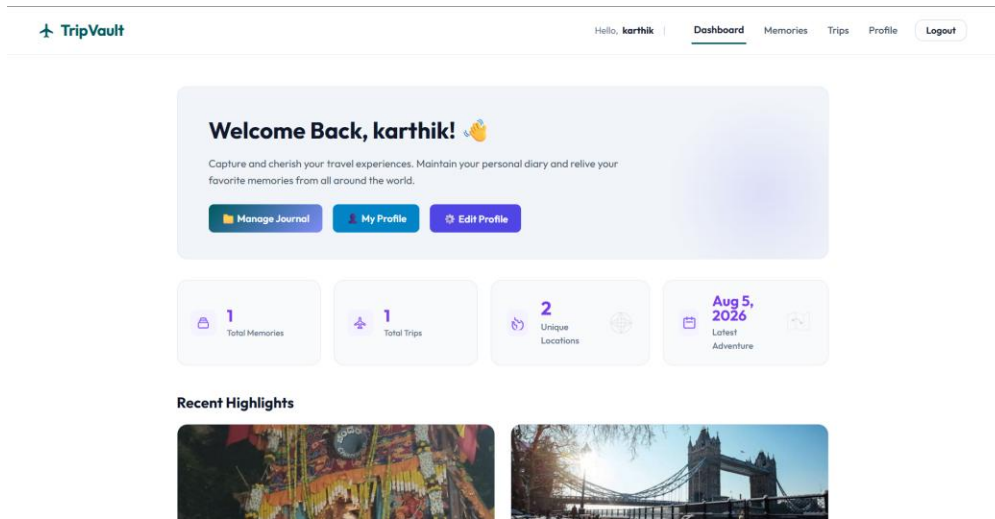


Figure 7: TripVault Frontend — Dashboard (My Profile / Edit Profile actions)

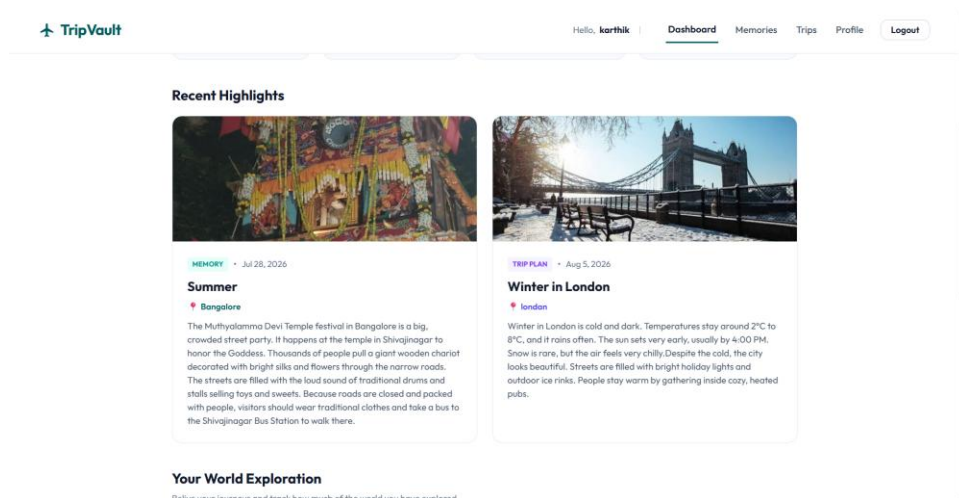


Figure 8: TripVault Frontend — Dashboard Recent Highlights

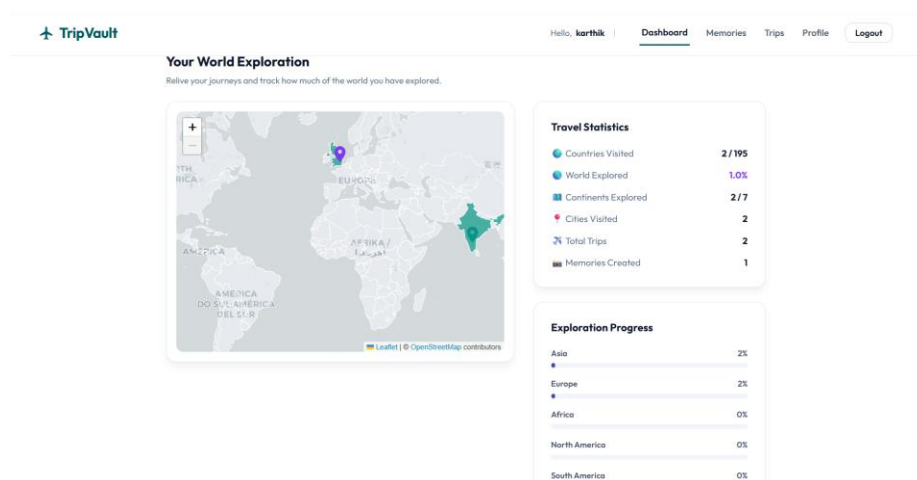


Figure 9: TripVault Frontend — World Exploration & Travel Statistics

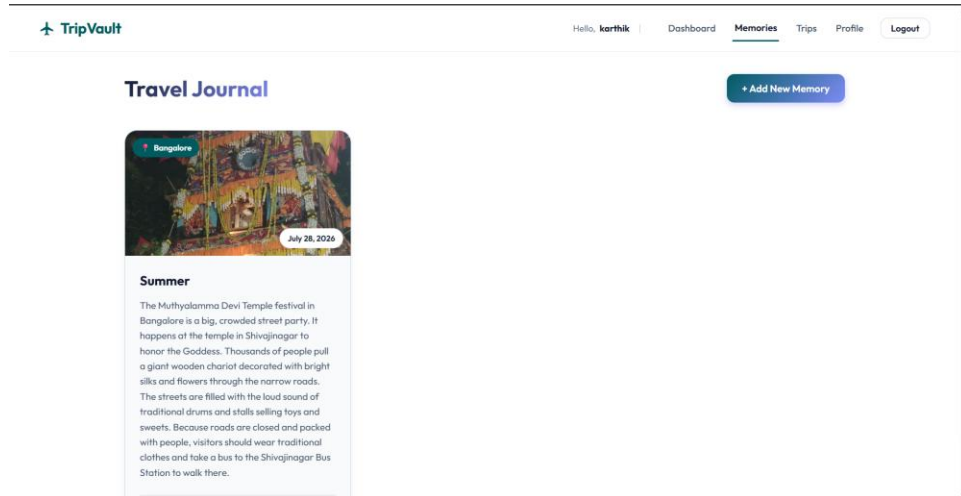


Figure 10: TripVault Frontend — Travel Journal (Memories)

The screenshot displays the 'New Memory' form, which is a modal window with a close button (X) in the top right corner. The form contains several input fields: 'Title' (with the placeholder 'Scuba diving in Maldives'), 'Location' (with the placeholder 'Malé, Maldives'), 'Country' (with the placeholder 'Maldives'), and 'City' (with the placeholder 'Malé'). Below these are 'Start Date' and 'End Date' fields, both set to '17-08-2026' with calendar icons. A 'Display Date (Journal Date)' field is also present, set to '17-08-2026' with a calendar icon. At the bottom is a 'Description / Diary Entry' field with the placeholder text 'Detail your experience, the food, people, and vibes...'. The form is styled with a light blue background and rounded corners.

Figure 11: TripVault Frontend — New Memory Form

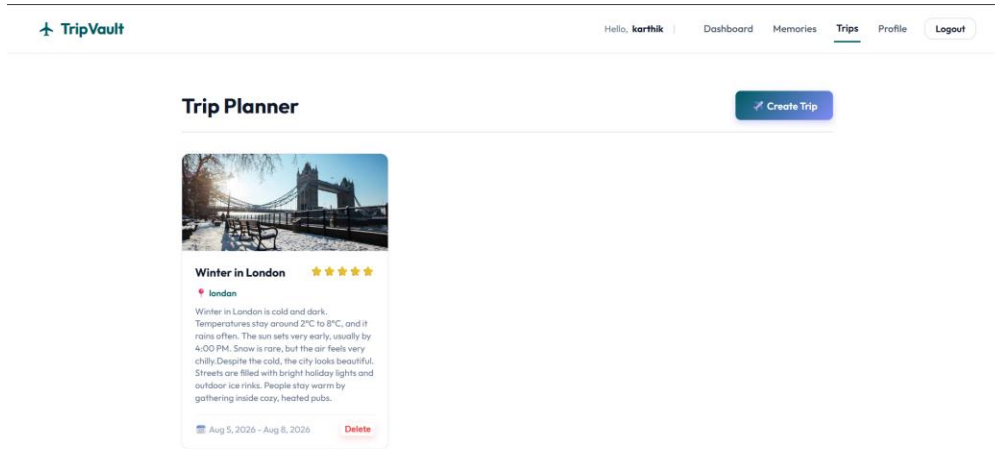


Figure 12: TripVault Frontend — Trip Planner (Trip List)

The screenshot shows the "Plan New Trip" form in the TripVault Frontend. The form is titled "Plan New Trip" and has a close button (X) in the top right corner. It contains several input fields and a rating section. The "Title" field is labeled "Title" and contains the text "Winter in Switzerland". The "Destination" field is labeled "Destination" and contains the text "Zurich, Switzerland". The "Start Date" field is labeled "Start Date" and contains the date "17-08-2026" with a calendar icon. The "End Date" field is labeled "End Date" and contains the date "17-08-2026" with a calendar icon. The "Rating (1 - 5)" section shows a 5-star rating (5/5) with a dropdown arrow. The "Trip Image" section has a "Choose file" button and the text "No file chosen". The "Description / Notes" section has a text area with the placeholder text "Write details about hotels, flights, or highlights...".

Figure 13: TripVault Frontend — Plan New Trip Form

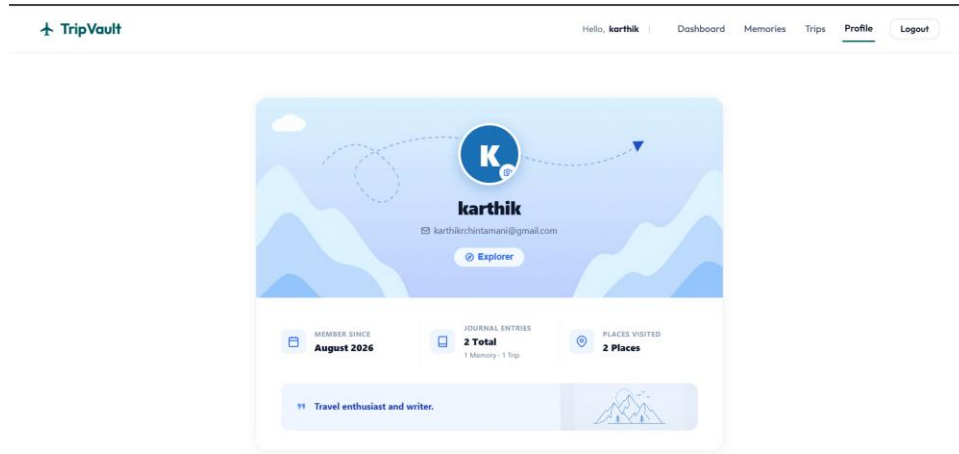


Figure 14: TripVault Frontend — Profile Page (bio, avatar upload, stats)

API Documentation (Swagger)

The full set of Users, Trip, Memory, and Authentication endpoints, documented and testable through Swagger UI:

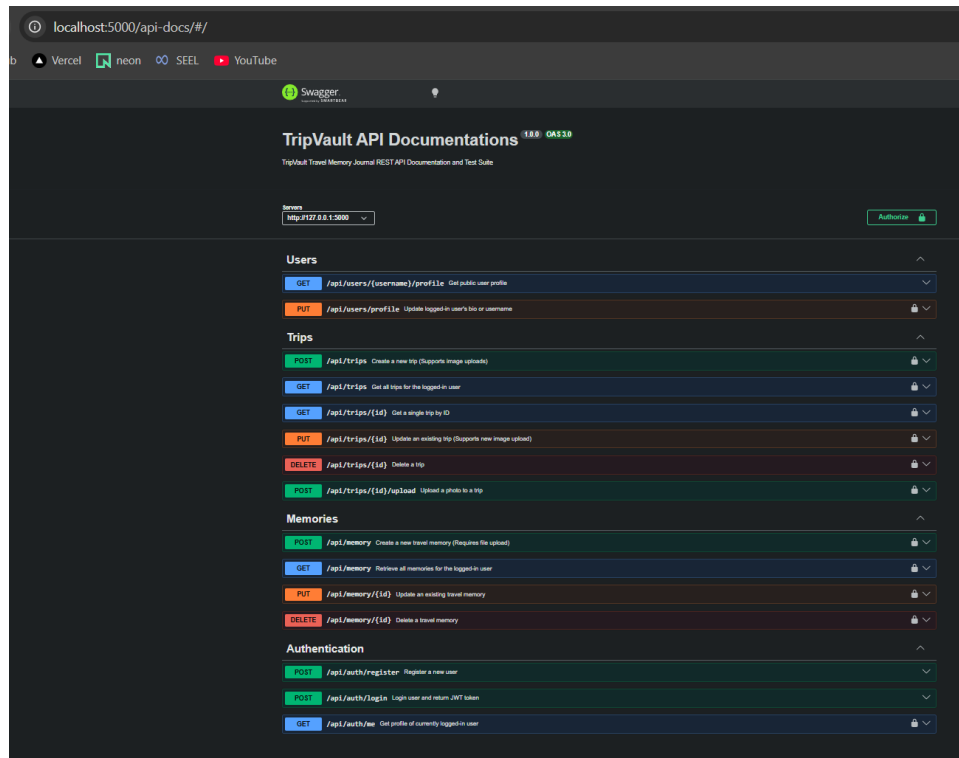


Figure 15: Swagger — API Endpoint Overview (Users, Trips, Memories, Auth)

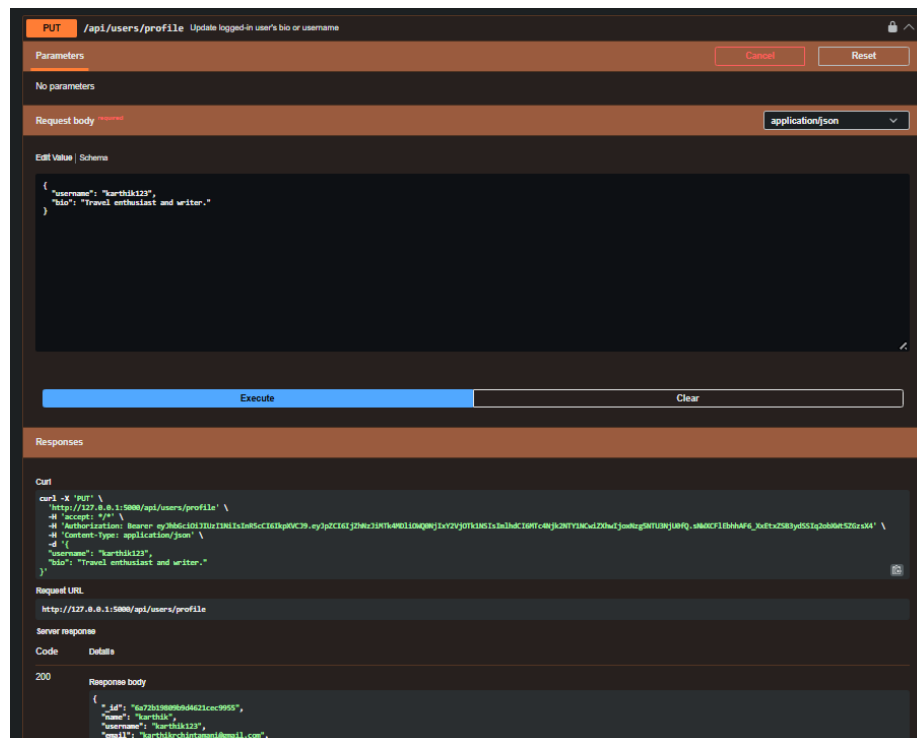


Figure 16: Swagger — PUT /api/users/profile (Update Username / Bio)

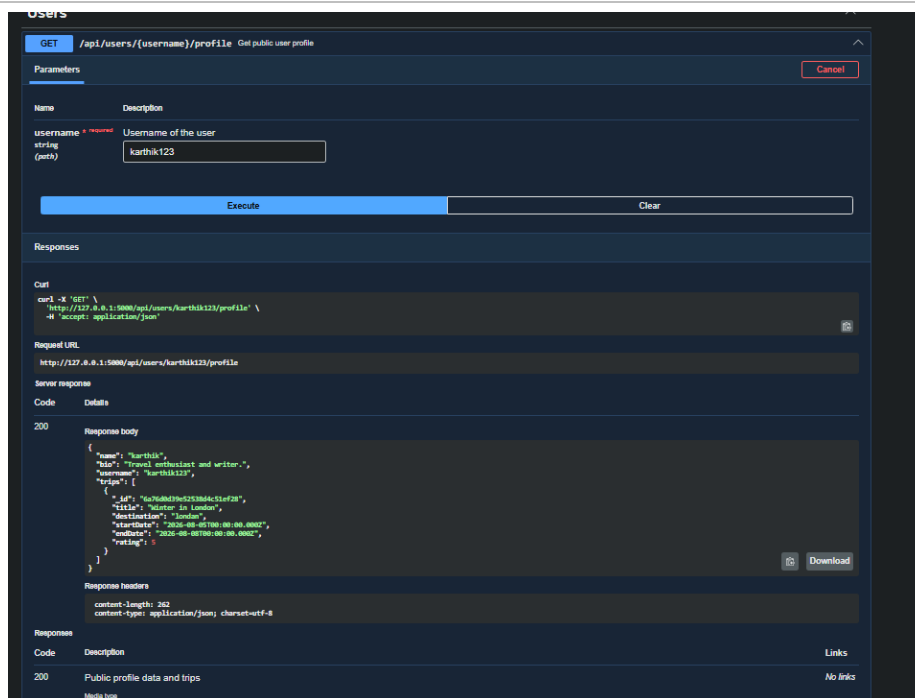


Figure 17: Swagger — GET /api/users/{username}/profile (Public Profile)

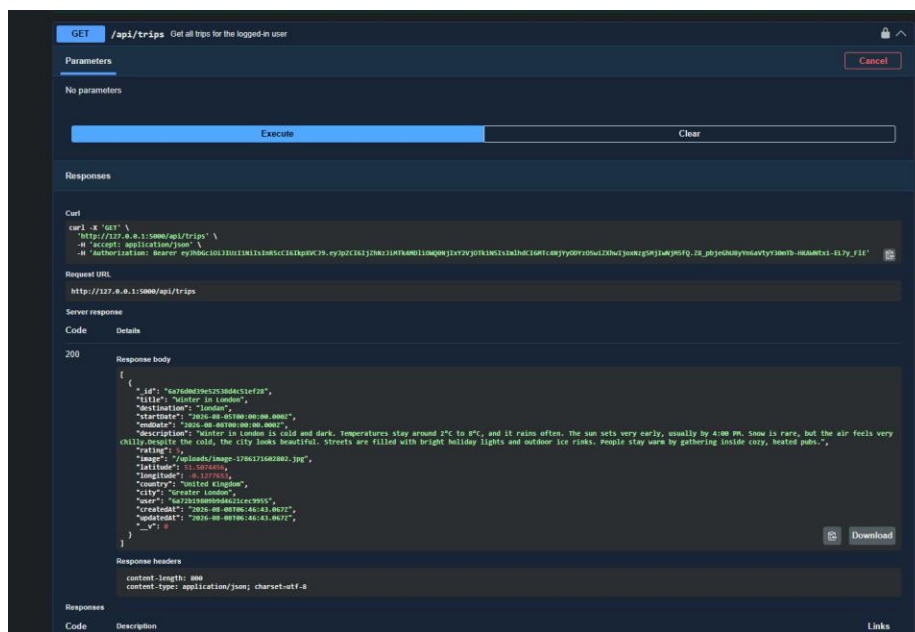
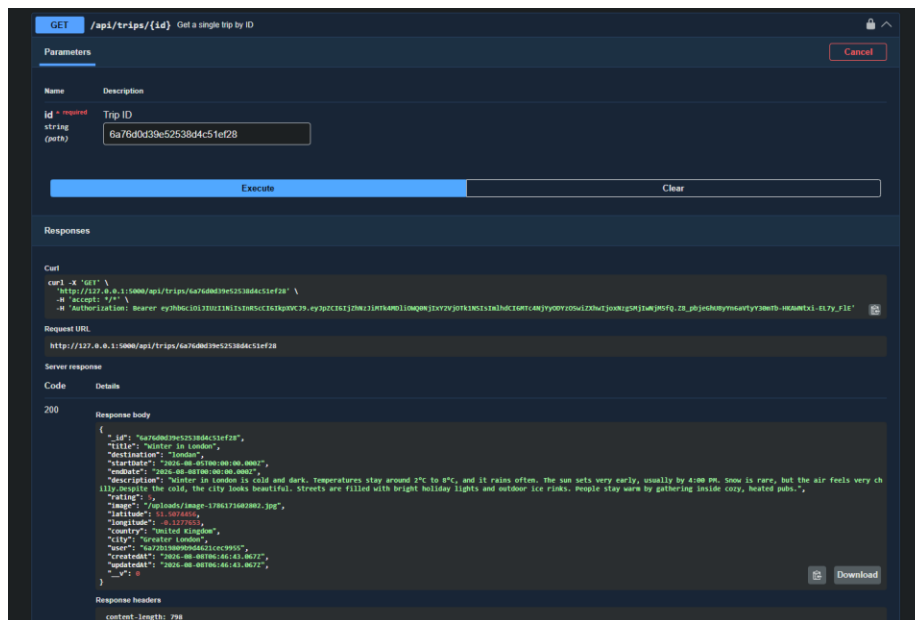
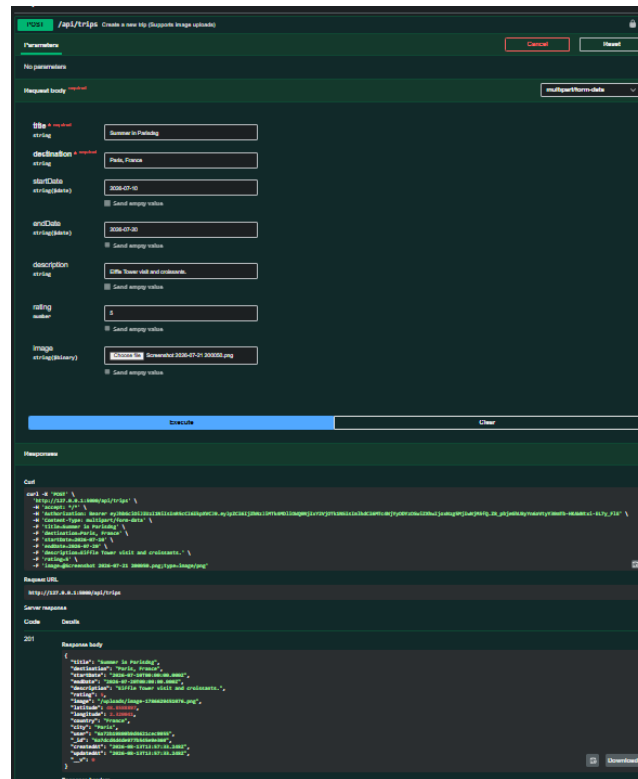


Figure 18: Swagger — GET /api/trips Response

Figure 19: Swagger — `GET /api/trips/{id}` ResponseFigure 20: Swagger — `POST /api/trips` (Create Trip)

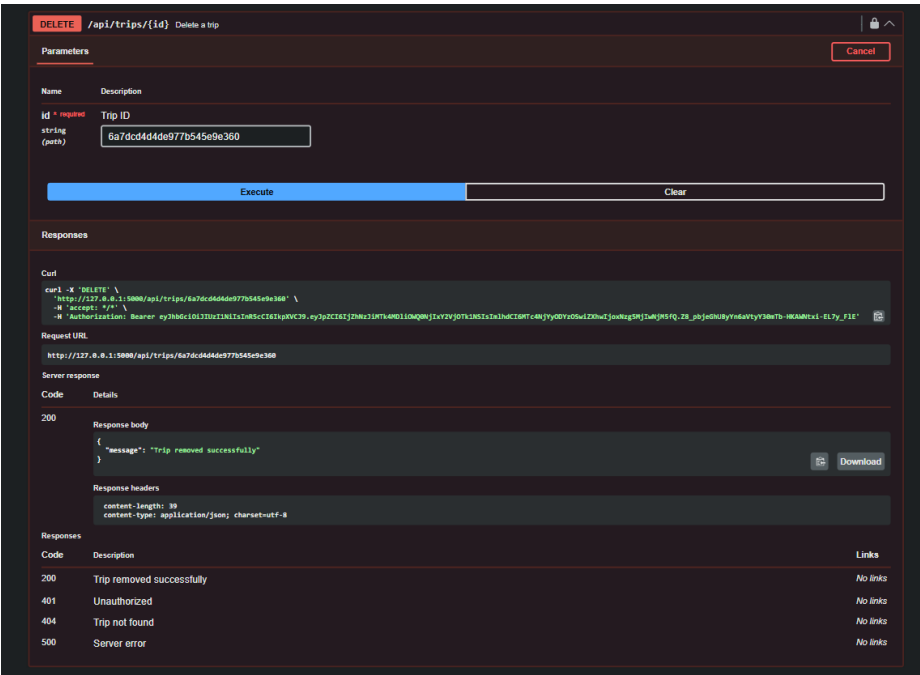


Figure 21: Swagger — DELETE /api/trips/{id} Response

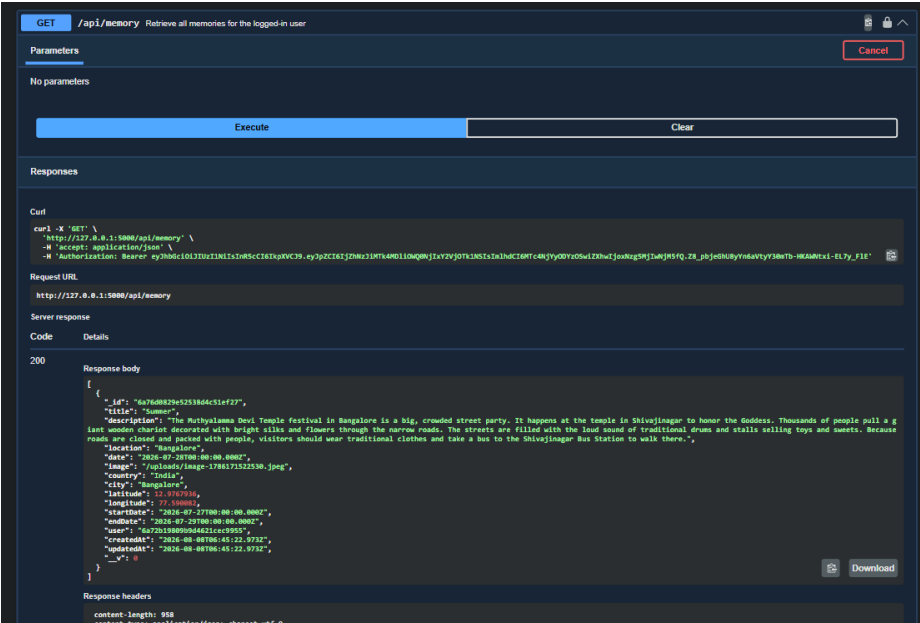


Figure 22: Swagger — GET /api/memory Response

POST

/api/memory

Create a new travel memory (Requires file upload)

Parameters

Cancel

Reset

No parameters

Request body

multipart/form-data

title *

required

string

Exploring Tokyo

description *

required

string

Visited Sensoji temple and ate ramen.

location *

required

string

Asakusa, Tokyo, Japan

date *

required

string(Min)

2024-08-01

country *

required

string

Japan

city *

required

string

Tokyo

startDate *

required

string(Min)

2024-08-01

endDate *

required

string(Min)

2024-08-05

image *

string(Binary)

Choose file

ScreenShot 2024-07-22 11:44:02.png

Send empty value

Execute

Clear

Responses

Curl

curl -X 'POST' \\\n -H 'Host: 127.0.0.1:3000/api/memory' \\\n -d '{\"accept\":\"*/*\", \"description\":\"Visited Sensoji temple and ate ramen.\", \"location\":\"Asakusa, Tokyo, Japan\", \"date\":\"2024-08-01\", \"country\":\"Japan\", \"city\":\"Tokyo\", \"startDate\":\"2024-08-01\", \"endDate\":\"2024-08-05\", \"image\":\"ScreenShot 2024-07-22 11:44:02.png\"}' \\\n -v

Request URL

http://127.0.0.1:3000/api/memory

Server response

Code

Details

201

Response body

{ \"title\": \"Exploring Tokyo\", \"description\": \"Visited Sensoji temple and ate ramen.\", \"location\": \"Asakusa, Tokyo, Japan\", \"date\": \"2024-08-01\", \"country\": \"Japan\", \"city\": \"Tokyo\", \"startDate\": \"2024-08-01\", \"endDate\": \"2024-08-05\", \"image\": \"ScreenShot 2024-07-22 11:44:02.png\"}

Figure 23: Swagger — POST /api/memory (Create Memory)

DELETE

/api/memory/{id} Delete a travel memory

Cancel

Name	Description
id * required	Memory document ID
string (path)	6a7dcde14de977b545e9e361

Execute

Clear

Responses

Curl

```
curl -X DELETE \
  "http://127.0.0.1:5000/api/memory/6a7dcde14de977b545e9e361" \
  -H "accept: */*" \
  -H "authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpcCI6IjE2MTU4MjE0MTQwMDIwOTY0Rk1hNiIsImh0bCI6ImFyZy9yb29zeWd3b29uZG9jaW4yNS91LmJpZG90b3R5bm9wZXI6ImNwZXI6IiJyIjoiIi" \
```

Request URL

http://127.0.0.1:5000/api/memory/6a7dcde14de977b545e9e361

Server response

Code	Details
200	Response body <pre>{ "message": "Memory removed successfully" }</pre>

Download

Figure 24: Swagger — DELETE /api/memory/{id} Response

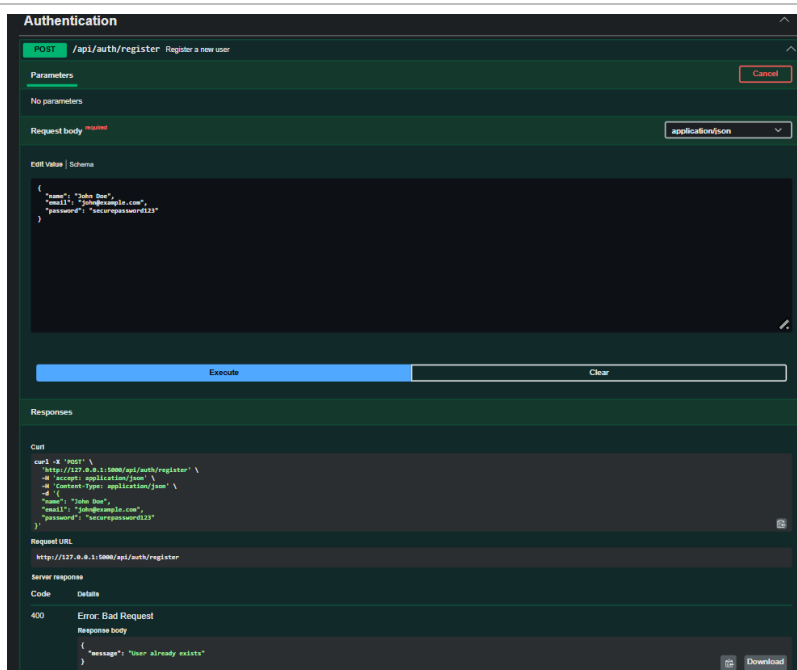


Figure 25: Swagger — POST /api/auth/register

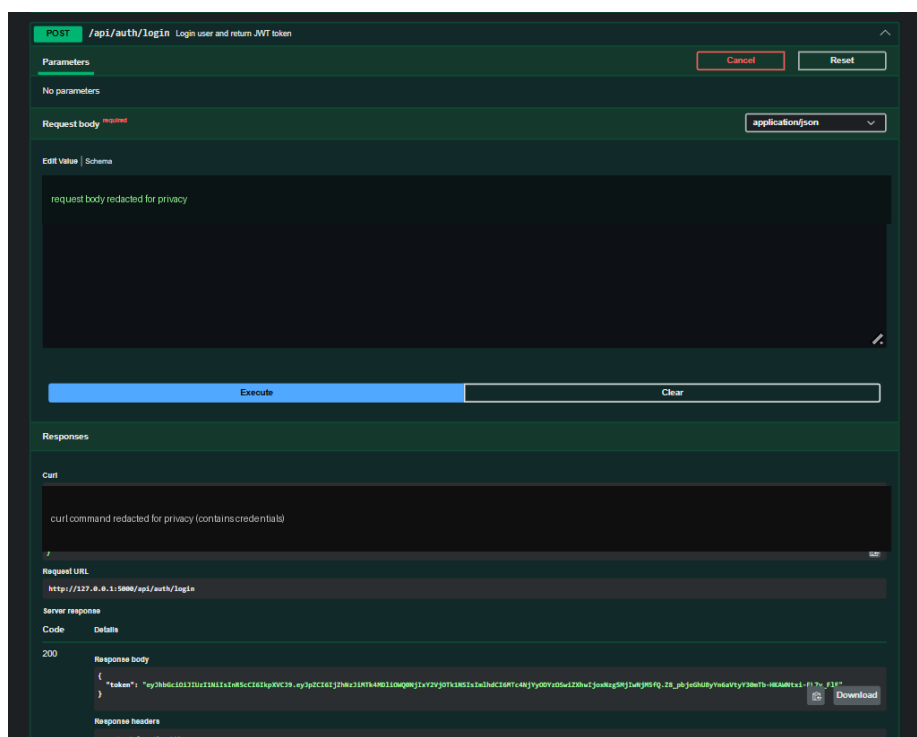


Figure 26: Swagger — `POST /api/auth/login` (credentials redacted)

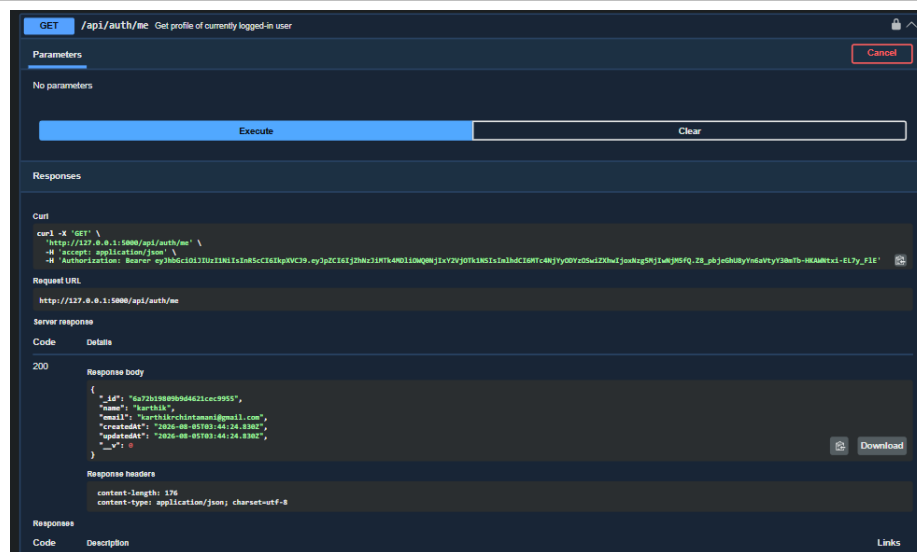


Figure 27: Swagger — GET /api/auth/me Response

Database

Trip and memory documents persisted in the MongoDB instance, viewed through MongoDB Compass. Trips shown below were created in Week 2, prior to the coverImage/photos schema addition; new and updated trips populate those fields via the upload route documented above.

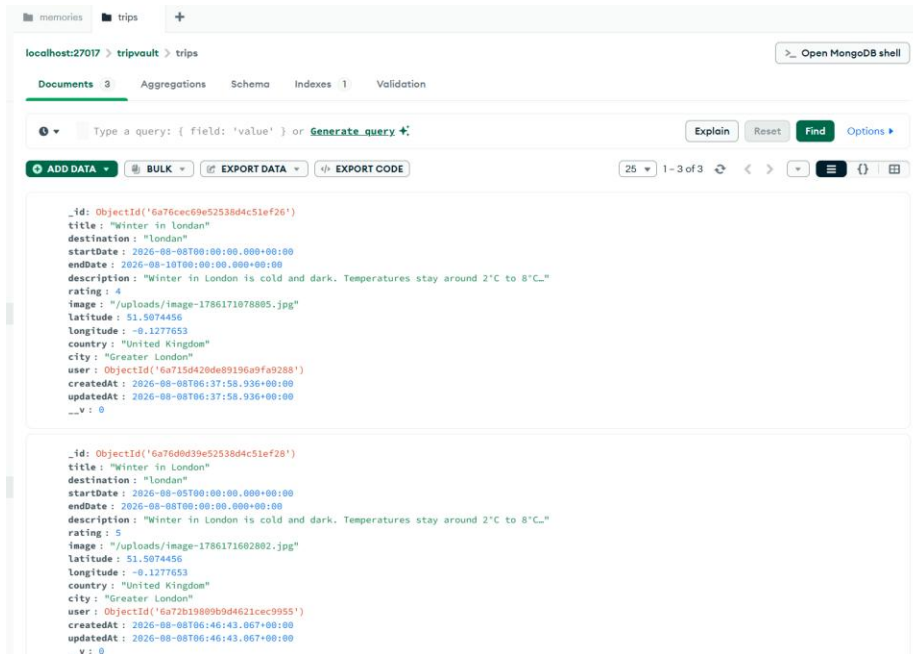


Figure 28: MongoDB — trips Collection

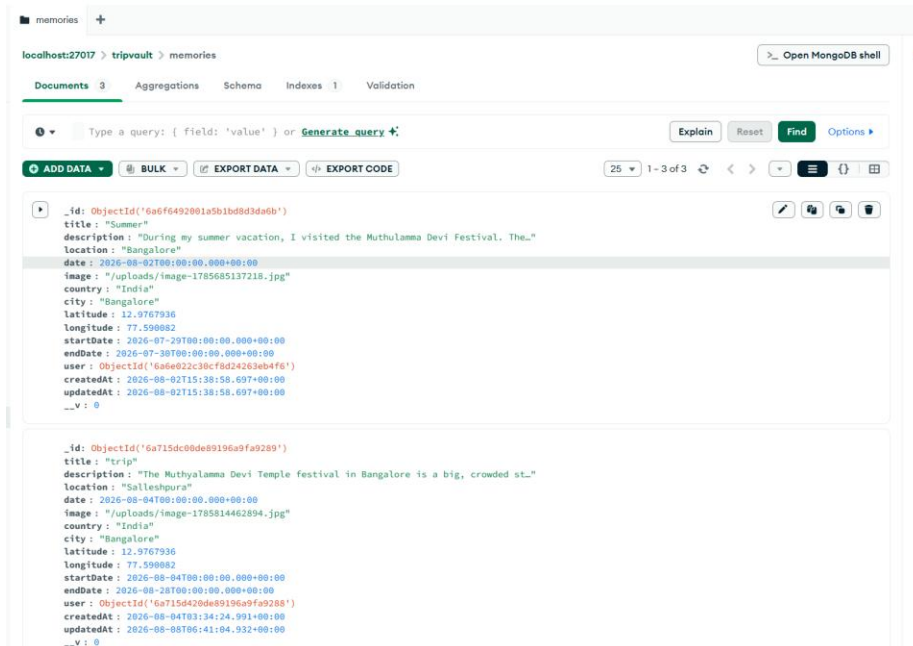


Figure 29: MongoDB — memories Collection

18. Algorithms

Username Auto-Generation Algorithm

1. Start.
2. Triggered when a user registers without a username, or logs in with an account that predates this field.
3. Take the portion of the user's email before the @ symbol as a base candidate.
4. Strip characters that are not letters or digits from the candidate.
5. Check whether a user with that username already exists.
6. If it does, append an incrementing numeric suffix and check again, repeating until a free username is found.
7. Save the generated username to the user's document.
8. End.

Public Profile Retrieval Algorithm

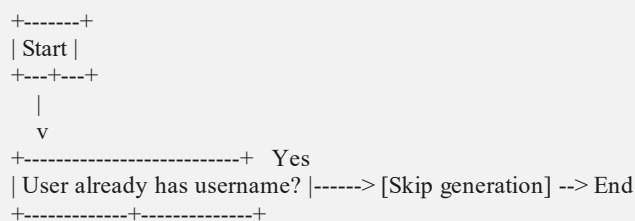
9. Start.
10. Receive a username from the URL path (no authentication required).
11. Query the User collection for a document with that username, selecting only name, bio, and username.
12. If no user is found, return a 404 error and stop.
13. Query the Trip collection for all trips belonging to that user, selecting only public-safe fields.
14. Return the combined name, bio, username, and trip list as JSON.
15. End.

Trip Photo Upload Algorithm

16. Start.
17. Receive a trip ID from the URL and an image file from the request.
18. Verify the request carries a valid JWT (authentication middleware).
19. Look up the trip by ID, filtered by the authenticated user's ID.
20. If no matching trip is found, return a 404 error and stop.
21. Let Multer stream the file to Cloudinary via the CloudinaryStorage engine.
22. Save the returned Cloudinary URL to the trip's coverImage field.
23. Return the updated trip to the client.
24. End.

19. Flowcharts

Username Auto-Generation Flowchart



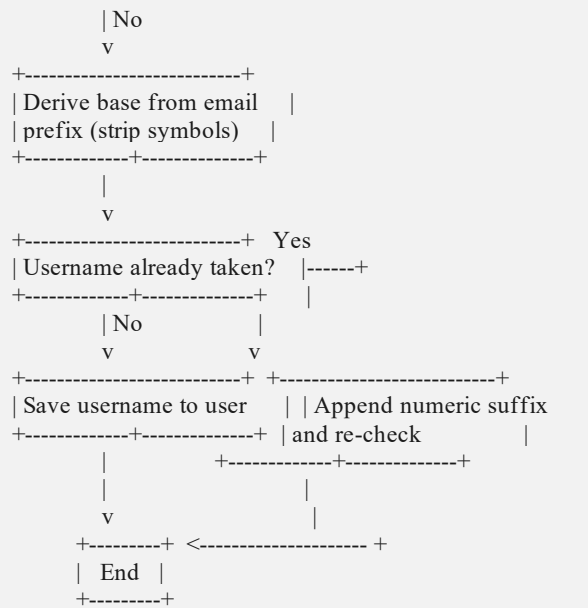


Figure 3: Username Auto-Generation Flowchart

Public Profile Retrieval Flowchart

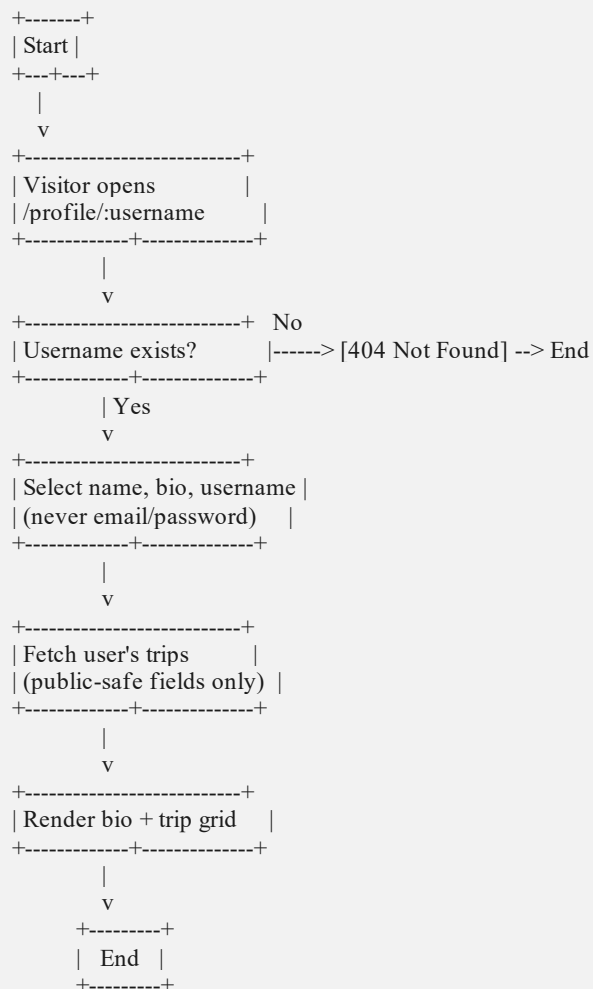


Figure 4: Public Profile Retrieval Flowchart

Trip Photo Upload Flowchart

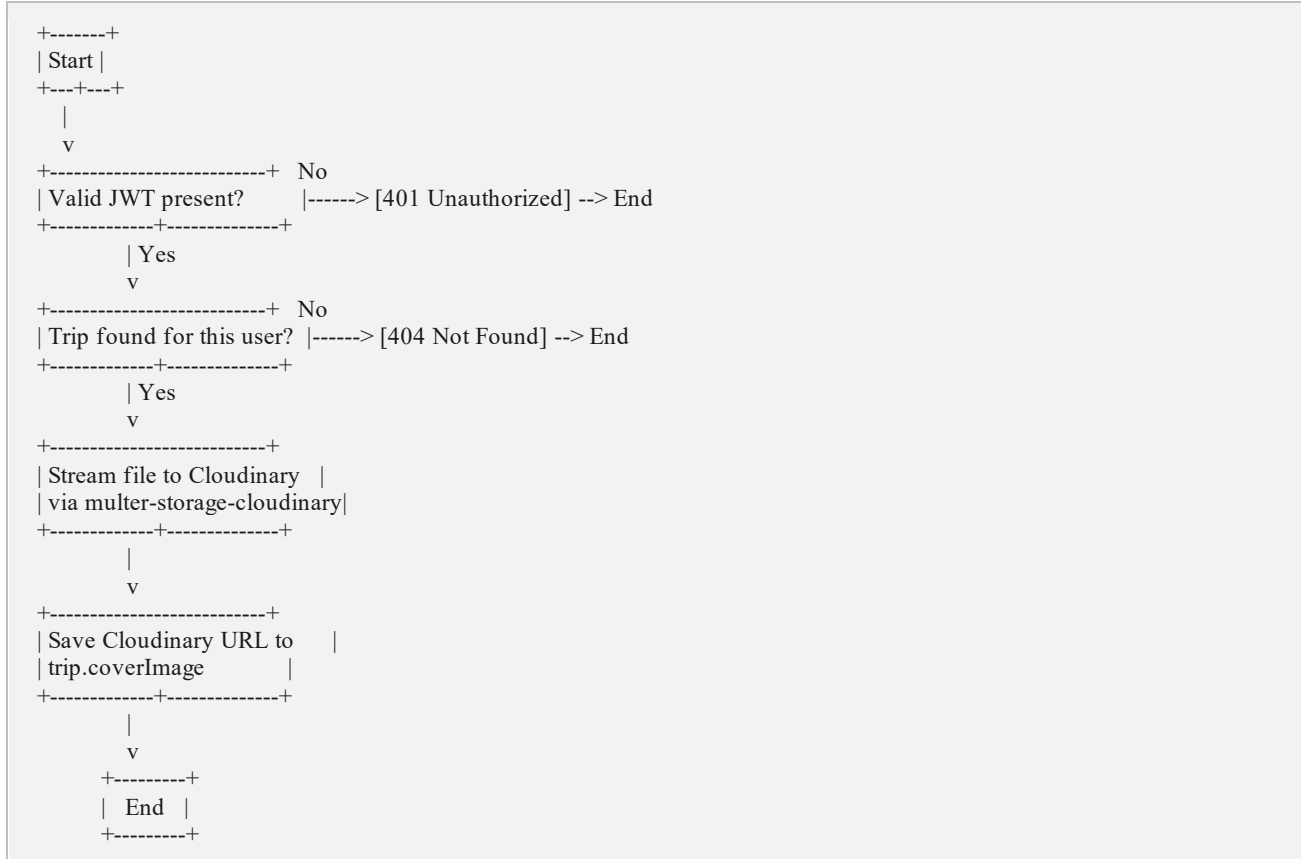


Figure 5: Trip Photo Upload Flowchart

20. Results

By the end of Week 3, trip photos uploaded through the API are stored on Cloudinary rather than the application server's disk, verified by inspecting the returned URLs and confirming the corresponding assets in the Cloudinary media library. Every pre-existing account from Weeks 1–2 was confirmed to receive a valid, unique, auto-generated username on next login, with no registration or login flow blocked by the new required-field constraint. The public profile endpoint was verified to return a complete, correct trip list for a valid username, a 404 for an invalid one, and — checked explicitly with a raw database comparison — no email or password field under any circumstance. The Dashboard's new My Profile and Edit Profile actions, and the standalone PublicProfile page, were confirmed working end-to-end, including viewing a profile in a logged-out browser session.

21. Advantages

- Cloud-hosted trip photos remove image storage and delivery from the application server's responsibilities entirely.
- The username backfill strategy adds a required, unique field without a disruptive migration or forcing existing users to re-register.

- Explicit field selection on the public profile query is a stronger privacy guarantee than filtering fields out after the fact.
- Splitting public and protected routes within the same `/api/users` group keeps related functionality together while keeping the security boundary clear.
- The `coverImage/photos` split on the Trip model allows a single-photo experience to ship now without blocking a multi-photo gallery later.

22. Future Enhancements

- Multi-Photo Trip Galleries — surface the photos array in the UI, letting a trip carry more than one image.
- Profile Avatars — extend Cloudinary uploads to user profile pictures, building on the avatar placeholder already shown on the Profile page.
- Interactive World Map — highlighting visited trip destinations geographically.
- Visited Countries Tracker — automatic tallying of countries and regions explored.
- Social Features — allow other users to follow a public profile or leave comments on a shared trip.
- Weather API Integration — historical or forecast weather data for trip destinations.
- AI Travel Recommendation — personalised destination suggestions based on trip history.
- Expense Tracker — logging and categorising trip-related expenses.
- Travel Analytics Dashboard — deeper statistics such as distance travelled and trip frequency over time.

23. Conclusion

Week 3 of the TripVault Virtual Internship Program has successfully layered cloud media storage and public profile sharing on top of the authentication and trip management foundations built in Weeks 1 and 2. With Cloudinary-backed photo uploads, a backfilled username system, a carefully scoped public profile endpoint, and a matching PublicProfile page all implemented and tested, TripVault now supports sharing a traveller's journeys beyond their own account — setting up naturally for richer social and gallery features in the milestones ahead.

24. References

React Documentation — <https://react.dev/>

Node.js Documentation — <https://nodejs.org/en/docs/>

Express.js Documentation — <https://expressjs.com/>

MongoDB Documentation — <https://www.mongodb.com/docs/>

Mongoose Documentation — <https://mongoosejs.com/docs/>

Cloudinary Node.js SDK Documentation — https://cloudinary.com/documentation/node_integration

multer-storage-cloudinary — <https://www.npmjs.com/package/multer-storage-cloudinary>

Multer Documentation — <https://github.com/expressjs/multer>

Swagger / OpenAPI Documentation — <https://swagger.io/docs/>

JWT (JSON Web Token) Documentation — <https://jwt.io/introduction>

25. Appendix

Sample JSON Request (Update Profile)

```
{
  "username": "karthik123",
  "bio": "Travel enthusiast and writer."
}
```

Sample JSON Response (Update Profile)

```
{
  "_id": "6a72b19809b9d4621cec9955",
  "name": "karthik",
  "username": "karthik123",
  "email": "karthikrchintamani@gmail.com",
  "bio": "Travel enthusiast and writer."
}
```

Sample JSON Response (Public Profile)

```
{
  "name": "karthik",
  "bio": "Travel enthusiast and writer.",
  "username": "karthik123",
  "trips": [
    {
      "id": "6a76d0d39e52538d4c51ef28",
      "title": "Winter in London",
      "destination": "london",
      "startDate": "2026-08-05T00:00:00.000Z",
      "endDate": "2026-08-08T00:00:00.000Z",
      "rating": 5
    }
  ]
}
```